evangelionPL

Manual del lenguaje. Fases 1 y 2

Juan Pablo Orjuela Rojas
Lenguajes de Programación y Transducción
Ciencias de la Computación e Inteligencia Artificial
Universidad Sergio Arboleda, 2026-2

1. Qué es evangelionPL

evangelionPL es un lenguaje de dominio específico para trabajar con tablas de datos: cargar un CSV, escoger columnas, filtrar filas, crear columnas calculadas, agrupar, sacar estadísticas y guardar el resultado. Es lo que pide el proyecto del curso, solo que las instrucciones se llaman como cosas de Neon Genesis Evangelion. Los datos entran por el `ENTRY_PLUG`, el `AT_FIELD` deja pasar únicamente las filas que cumplen una condición y las `MAGI` hacen los cálculos.

La idea central es el flujo: se parte de una tabla y se le encadenan etapas con `|>`. Cada etapa recibe una tabla y entrega otra nueva, así que el programa se lee de arriba hacia abajo como una lista de pasos. El lenguaje no pretende ser otro Python: solo sabe hacer estas operaciones, pero las dice corto.

Está hecho con ANTLR4 y Python. Este documento cubre las dos primeras fases del proyecto.

Fase	Qué pide el curso	Estado
1	Especificación del lenguaje, gramática, lexer y parser, árbol de análisis y errores léxicos y sintácticos.	Completa
2	Visitor, tabla de símbolos, ejecución sobre datos con pandas, agregaciones, exportar a CSV y errores semánticos.	Completa
3	Gráficas y caso de estudio final.	Pendiente. `HOLOGRAM` ya se reconoce y se revisa, pero no dibuja.

Un primer programa

```
# Mi primer programa: cargar las pruebas y ver las cinco mejores
NERV "Primera prueba de sincronizacion"

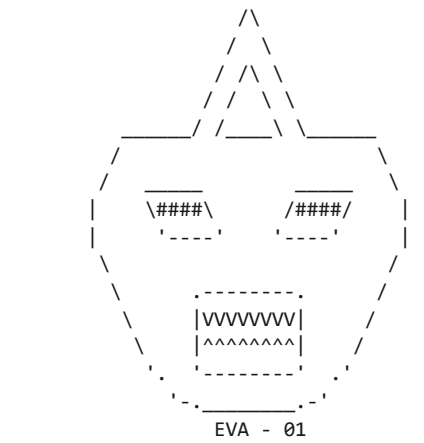
pruebas = ENTRY_PLUG "datos/pruebas_sincronizacion.csv"

mejores = pruebas
    |> TARGET [fecha, piloto, unidad, sincronizacion]
    |> AT_FIELD donde sincronizacion >= 60 y piloto != "Kaworu"
    |> FORMATION por [sincronizacion desc]
    |> BATTERY 5

MONITOR mejores
```

Al ejecutarlo con `python run.py ejemplos/01_primera_prueba.eva` sale la tabla y, como todo estuvo bien, el EVA-01 felicita:

```
MONITOR mejores (5 filas x 4 columnas)
     fecha piloto unidad  sincronizacion
2015-10-19 Shinji EVA-01           93.50
2015-07-20  Asuka EVA-02           92.70
2015-08-03  Asuka EVA-02           86.60
2015-10-05 Shinji EVA-01           80.10
2015-09-07    Rei EVA-00           77.90
```




```
¡Felicidades, Shinji! (omedetou!)
Sincronización con el EVA-01 al 100 %: 01_primera_prueba.eva corrió sin errores.

  Operación : Primera prueba de sincronizacion
  Tokens    : 42
  Variables : pruebas, mejores
```

2. Catálogo de instrucciones

Las instrucciones van en mayúscula y son términos de la serie. Los conectores (`donde`, `por`, `como`, `con`, `de`) van en minúscula y en español. La tercera columna cuenta por qué cada instrucción se llama así.

Instrucción	Qué hace	Por qué ese nombre
`NERV "título"`	Le pone nombre a la operación. Es opcional y va en la primera línea.	NERV es la organización que dirige todo.
`ENTRY_PLUG "archivo.csv"`	Carga un archivo CSV como tabla.	Es la cápsula por donde el piloto entra al EVA; aquí es por donde entran los datos.
`TARGET [a, b]`	Deja solo las columnas de la lista.	Fijar el blanco antes de disparar.
`AT_FIELD donde condición`	Deja pasar solo las filas que cumplen la condición.	El campo AT es la barrera que no deja pasar casi nada.
`SYNC col = expresión`	Crea una columna calculada (o varias, separadas por coma).	La sincronización entre piloto y EVA.
`CODENAME [vieja -> nueva]`	Les cambia el nombre a las columnas.	Los nombres clave.
`FORMATION por [col desc]`	Ordena las filas; cada columna puede ir asc o desc.	La formación en la que salen las unidades.

Instrucción	Qué hace	Por qué ese nombre
`EJECT duplicados | faltantes | columnas`	Saca filas repetidas, filas con vacíos o columnas completas.	La eyección de la cápsula.
`DUMMY_PLUG col = valor`	Rellena los vacíos de una columna.	El dummy plug reemplaza al piloto cuando falta.
`EQUIP col -> tipo`	Convierte la columna a entero, real, texto o booleano.	Los equipos (tipo B, tipo D) que le cambian al EVA.
`SQUAD por [a, b]`	Arma los grupos. Después siempre va MAGI.	El escuadrón.
`MAGI n = contar(), m = media(col)`	Calcula un resumen por grupo, o de toda la tabla si no hay grupos.	Las supercomputadoras que hacen todos los cálculos de NERV.
`BATTERY n`	Deja las primeras n filas.	La batería interna, que dura poquito.
`DIAGNOSTIC`	Estadísticas de cada columna: válidos, faltantes, distintos, media, desviación, mínimo y máximo.	El chequeo de la unidad.
`MONITOR flujo`	Muestra una tabla o un valor en la terminal.	Las pantallas del centro de mando.
`SEELE flujo como "archivo.csv"`	Guarda una tabla en un CSV.	El comité al que NERV le entrega informes.
`PATTERN_BLUE cond { } PATTERN_ORANGE { }`	Condicional: ejecuta el primer bloque si se cumple y, si no, el segundo.	Patrón azul es ángel confirmado; patrón naranja, sin identificar.
`HOLOGRAM tipo de tabla { }`	Describe una gráfica. Por ahora solo se revisa; se dibuja en la fase 3.	Los hologramas de las reuniones de SEELE.

Funciones

Función	Uso
`suma media mediana desviacion`	Agregaciones sobre una columna numérica.
`minimo maximo contar`	Agregaciones sobre cualquier columna. `contar()` sin columna cuenta filas.
`filas(tabla)`	Cuántas filas tiene una tabla.
`redondear(x, n)`, `absoluto(x)`	Para números o columnas numéricas.
`mayusculas(t)`, `minusculas(t)`	Para textos o columnas de texto.

Los tipos de `EQUIP` son entero, real, texto y booleano. Los tipos de `HOLOGRAM` son barras, lineas, histograma, dispersion y caja. Ninguno es palabra reservada: se validan en la fase semántica, para que una columna pueda llamarse, por ejemplo, `texto`.

3. Fase 1: análisis léxico

El lexer está en `grammar/EvaLexer.g4`. Lee el archivo carácter por carácter y entrega tokens. Estas tablas se generan a partir del lexer que produce ANTLR, así que siempre coinciden con la gramática.

Categoría	Tokens (nombre y lexema)
Instrucciones	`NERV`, `ENTRY_PLUG`, `MONITOR`, `SEELE`, `HOLOGRAM`, `PATTERN_BLUE`, `PATTERN_ORANGE`
Etapas del flujo	`TARGET`, `AT_FIELD`, `SYNC`, `CODENAME`, `FORMATION`, `EJECT`, `DUMMY_PLUG`, `EQUIP`, `SQUAD`, `MAGI`, `BATTERY`, `DIAGNOSTIC`
Conectores	`DONDE donde`, `POR por`, `COMO como`, `CON con`, `DE de`, `ASC asc`, `DESC desc`, `DUPLICADOS duplicados`, `FALTANTES faltantes`, `COLUMNAS columnas`
Lógicos y literales fijos	`Y y`, `O o`, `NO no`, `VERDADERO verdadero`, `FALSO falso`, `NULO nulo`
Operadores y signos	`TUBERIA |>`, `FLECHA ->`, `IGUAL_IGUAL ==`, `DIFERENTE !=`, `MENOR_IGUAL <=`, `MAYOR_IGUAL >=`, `MENOR <`, `MAYOR >`, `IGUAL =`, `MAS +`, `MENOS -`, `POR_OP *`, `DIV /`, `MODULO %`, `POTENCIA ^`, `PAR_IZQ (`, `PAR_DER )`, `COR_IZQ [`, `COR_DER ]`, `LLAVE_IZQ {`, `LLAVE_DER }`, `COMA ,`, `DOS_PUNTOS :`, `PUNTO .`

Token	Qué reconoce
`REAL`	dígitos, un punto y más dígitos: 0.75
`ENTERO`	uno o más dígitos: 42
`CADENA`	texto entre comillas dobles: "Tokio-3"
`CADENA_SIN_CERRAR`	comilla que nunca se cierra; se reporta como error léxico
`ID`	letra o guion bajo seguido de letras, dígitos o guiones bajos: sync_01
`ID_ESCAPADO`	cualquier texto entre acentos graves: `` `Unidad EVA` ``
`COMENTARIO`	desde # hasta el final de la línea; se descarta
`ESPACIOS`	espacios, tabulaciones y saltos de línea; se descartan
`CARACTER_INVALIDO`	cualquier otro carácter; se reporta como error léxico

Decisiones del lexer

Las palabras reservadas se declaran antes que `ID`. Cuando dos reglas reconocen el mismo texto, ANTLR se queda con la primera, y por eso `MAGI` sale como instrucción y no como nombre. Si el texto es más largo (`MAGI_2`) gana `ID`, porque siempre se prefiere el lexema más largo. Las reservadas distinguen mayúsculas: `magi` es un nombre cualquiera.

`REAL` va antes que `ENTERO` para que `3.5` no se parta en tres tokens. Los comentarios y los espacios se descartan con `-> skip` y el parser nunca los ve.

Las palabras `y`, `o` y `no` son operadores, así que no sirven como nombres de variable. Para una columna que se llame así, o que tenga espacios, se usan acentos graves: `` `Unidad EVA` ``.

Hay dos tokens que existen solo para atrapar errores: `CARACTER_INVALIDO` y `CADENA_SIN_CERRAR`. Con ellos el lexer nunca se queda atascado y el error se reporta con la línea y la columna exactas.

Los tokens de un programa

Con `--tokens` se ve lo que entrega el lexer. Estos son los primeros del ejemplo anterior:

```
TOKENS de 01_primera_prueba.eva
   #  LIN:COL  TIPO               LEXEMA
   1  2:1      NERV               NERV
   2  2:6      CADENA             "Primera prueba de sincronizacion"
   3  4:1      ID                 pruebas
   4  4:9      IGUAL              =
   5  4:11     ENTRY_PLUG         ENTRY_PLUG
   6  4:22     CADENA             "datos/pruebas_sincronizacion.csv"
   7  6:1      ID                 mejores
   8  6:9      IGUAL              =
   9  6:11     ID                 pruebas
  10  7:5      TUBERIA            |>
  11  7:8      TARGET             TARGET
  12  7:15     COR_IZQ            [
  13  7:16     ID                 fecha
  14  7:21     COMA               ,
 ...
```

4. Fase 1: análisis sintáctico

El parser está en `grammar/EvaParser.g4`. Esta es la gramática en EBNF:

```
(* Gramatica de evangelionPL en EBNF.
   { x } = cero o mas veces, [ x ] = opcional, | = alternativa *)

programa       = [ cabecera ] , { sentencia } ;
cabecera       = "NERV" , CADENA ;

sentencia      = asignacion | mostrar | guardar | graficar | condicional ;
asignacion     = nombre , "=" , flujo ;
mostrar        = "MONITOR" , flujo ;
guardar        = "SEELE" , flujo , "como" , CADENA ;
graficar       = "HOLOGRAM" , ID , "de" , flujo , [ "{" , { opcion } , "}" ] ;
condicional    = "PATTERN_BLUE" , expresion , bloque , [ "PATTERN_ORANGE" , bloque ] ;
bloque         = "{" , { sentencia } , "}" ;

flujo          = fuente , { "|>" , etapa } ;
fuente         = carga | expresion ;
carga          = "ENTRY_PLUG" , CADENA , [ "con" , "{" , { opcion } , "}" ] ;
opcion         = ID , ":" , expresion , [ "," ] ;

etapa          = "TARGET" , lista_columnas
               | "AT_FIELD" , "donde" , expresion
               | "SYNC" , columna_nueva , { "," , columna_nueva }
               | "CODENAME" , "[" , renombre , { "," , renombre } , "]"
               | "FORMATION" , "por" , "[" , criterio , { "," , criterio } , "]"
               | "EJECT" , "duplicados" , [ lista_columnas ]
               | "EJECT" , "faltantes" , [ lista_columnas ]
               | "EJECT" , "columnas" , lista_columnas
               | "DUMMY_PLUG" , columna_nueva , { "," , columna_nueva }
               | "EQUIP" , nombre , "->" , ID
               | "SQUAD" , "por" , lista_columnas
               | "MAGI" , agregacion , { "," , agregacion }
               | "BATTERY" , ENTERO
               | "DIAGNOSTIC" ;

columna_nueva  = nombre , "=" , expresion ;
```

```
agregacion     = nombre , "=" , ID , "(" , [ nombre ] , ")" ;
renombre       = nombre , "->" , nombre ;
criterio       = nombre , [ "asc" | "desc" ] ;
lista_columnas = "[" , nombre , { "," , nombre } , "]" ;

(* Expresiones: cada nivel amarra mas fuerte que el anterior *)
expresion      = disyuncion ;
disyuncion     = conjuncion , { "o" , conjuncion } ;
conjuncion     = negacion , { "y" , negacion } ;
negacion       = "no" , negacion | igualdad ;
igualdad       = comparacion , { ( "==" | "!=" ) , comparacion } ;
comparacion    = suma , { ( "<" | "<=" | ">" | ">=" ) , suma } ;
suma           = termino , { ( "+" | "-" ) , termino } ;
termino        = unario , { ( "*" | "/" | "%" ) , unario } ;
unario         = "-" , unario | potencia ;
potencia       = primario , [ "^" , unario ] ;
primario       = literal
               | nombre
               | nombre , "." , nombre
               | ID , "(" , [ expresion , { "," , expresion } ] , ")"
               | "(" , expresion , ")" ;

literal        = ENTERO | REAL | CADENA | "verdadero" | "falso" | "nulo" ;
nombre         = ID | ID_ESCAPADO ;

(* Lexico *)
ENTERO         = digito , { digito } ;
REAL           = digito , { digito } , "." , digito , { digito } ;
CADENA         = '"' , { caracter } , '"' ;
ID             = letra , { letra | digito | "_" } ;
ID_ESCAPADO    = "`" , caracter , { caracter } , "`" ;
comentario     = "#" , { caracter } ;
```

Precedencia de los operadores

Nivel	Operadores	Asociatividad
1 (más fuerte)	`( )`, llamadas, `tabla.columna`	—
2	`^`	derecha
3	`-` de signo	—
4	`* / %`	izquierda
5	`+ -`	izquierda
6	`< <= > >=`	izquierda
7	`== !=`	izquierda
8	`no`	—
9	`y`	izquierda
10 (más débil)	`o`	izquierda

En la EBNF la precedencia se ve por niveles (suma, término, potencia…). En el archivo `.g4` es una sola regla `expresion` recursiva por la izquierda, y ANTLR saca la precedencia del orden de las alternativas: la que está más arriba amarra más fuerte. Las dos formas describen lo mismo, con una diferencia pequeña: el `.g4` también acepta un `no` como operando (`a == no b`). La potencia es asociativa a la derecha, así que `2 ^ 3 ^ 2` es 2 ^ 9, y `-2 ^ 2` da -4 como en matemáticas.

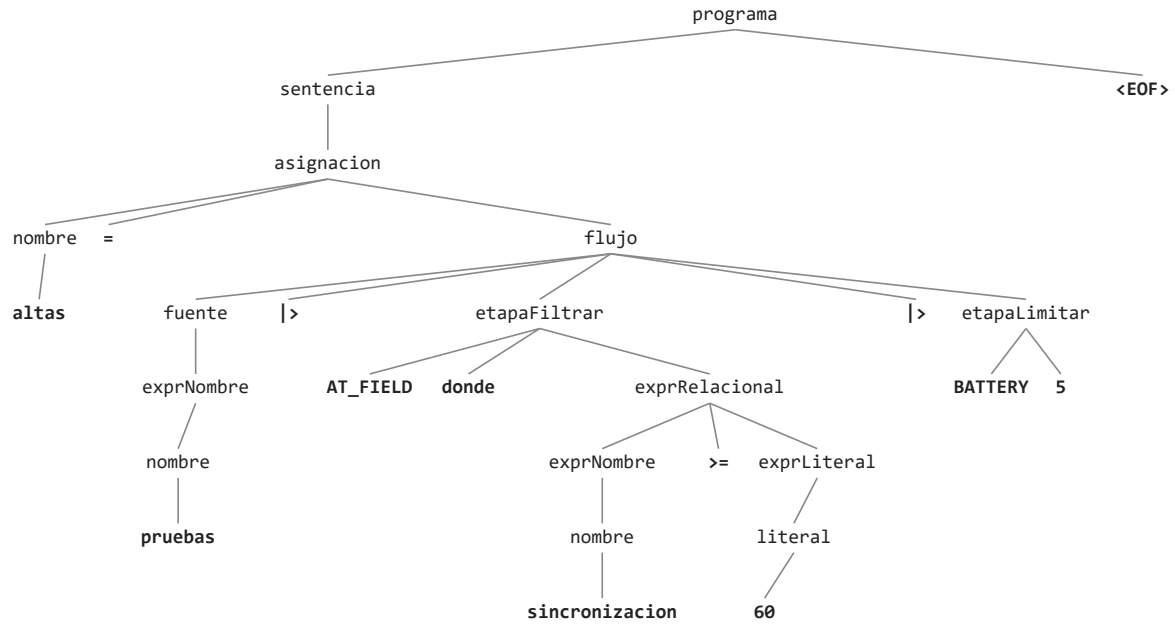
No hay punto y coma. Cada sentencia empieza con una instrucción o con `nombre =`, y con eso el parser sabe dónde termina la anterior. La regla `programa` termina en `EOF` para que no se pueda colar basura al final del archivo.

Árbol de análisis

Para esta línea:

```
altas = pruebas |> AT_FIELD donde sincronizacion >= 60 |> BATTERY 5
```

el parser arma el siguiente árbol. Se ve que la comparación quedó colgando de la etapa `AT_FIELD` y que las dos etapas son hermanas dentro del mismo flujo.



Árbol generado con `python run.py archivo.eva --svg arbol.svg`

El mismo árbol en texto, con `--arbol`:

```
programa
|-- sentencia
|   `-- asignacion
|       |-- nombre
|       |   `-- altas
|       |-- =
|       `-- flujo
|           |-- fuente
|           |   `-- exprNombre
|           |       `-- nombre
|           |           `-- pruebas
|           |-- |>
|           |-- etapaFiltrar
|           |   |-- AT_FIELD
|           |   |-- donde
|           |   `-- exprRelacional
|           |       |-- exprNombre
|           |       |   `-- nombre
|           |       |       `-- sincronizacion
|           |       |-- >=
|           |       `-- exprLiteral
|           |           `-- literal
|           |               `-- 60
|           |-- |>
|           `-- etapaLimitar
|               |-- BATTERY
|               `-- 5
`-- <EOF>
```

Los nodos llevan el nombre de la alternativa (`etapaFiltrar`, `exprRelacional`) y no solo el de la regla. Esas etiquetas son las que después usa el Visitor: ANTLR genera un método `visit` por cada una.

5. Fase 2: semántica y ejecución

Si el programa pasa la fase 1, el árbol se recorre con el patrón Visitor (`src/evangelion/interprete.py`). La clase `Interprete` hereda del `EvaParserVisitor` que genera ANTLR y tiene un método por cada tipo de nodo: `visitEtapaFiltrar`, `visitExprAditiva`, `visitGuardar`... Cada método revisa que lo que le llegó tenga sentido y después hace el trabajo con pandas.

```
programa .eva → lexer → tokens → parser → árbol → Visitor (revisa y ejecuta) → pandas → tablas y CSV
```

Tabla de símbolos

Cada asignación guarda en la tabla de símbolos el nombre, la clase del valor, el valor y la línea donde se definió. Hay un solo ámbito global: lo que se asigna dentro de un `PATTERN_BLUE` sigue existiendo afuera. Con `--simbolos` se imprime al terminar; así queda después del ejemplo 04:

```
TABLA DE SÍMBOLOS
  NOMBRE             CLASE     LÍNEA  VALOR
  pruebas            tabla     4      29 filas x 7 columnas
  asuka              tabla     8      8 filas x 4 columnas
  ultimas            tabla     17     3 filas x 4 columnas
  promedio_total     numero    19     66.6
  promedio_reciente  numero    20     47.9
  alerta             tabla     25     3 filas x 5 columnas
```

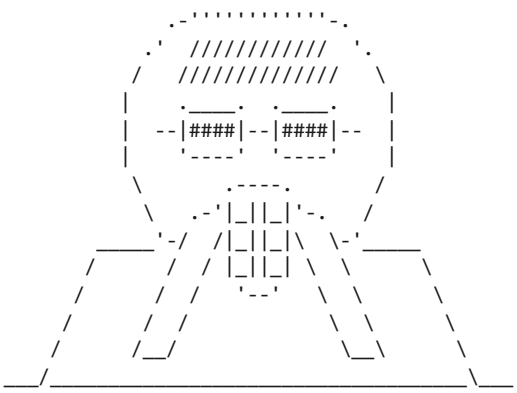
Las clases de valor son tabla, numero, texto, booleano y nulo. Una columna suelta (`tabla.columna`) se puede usar dentro de una expresión, pero no guardar en una variable.

Reglas semánticas

Sobre	Regla
Variables	Una variable tiene que estar asignada antes de usarse. Se puede volver a asignar.
Columnas	Toda columna que se nombre en una etapa tiene que existir en la tabla que llega. Si no existe, el error sugiere la que más se parece.
Nombres dentro de una etapa	Primero se busca entre las columnas de la tabla y después entre las variables.
Aritmética	+ - * / % ^ solo aceptan números. El + también pega dos textos. No se divide por cero.
Comparaciones	< <= > >= comparan número con número o texto con texto. == y != piden la misma clase a los dos lados, salvo que uno sea nulo.
Lógica	y, o, no solo aceptan valores de verdadero/falso.
Condiciones	La condición de AT_FIELD y la de PATTERN_BLUE tienen que dar verdadero o falso.
Etapas	Una etapa solo se aplica a una tabla. Ninguna cambia la tabla original: devuelve una nueva.
Grupos	Después de SQUAD la única etapa permitida es MAGI.
Agregaciones	suma, media, mediana y desviacion piden una columna numérica; minimo, maximo y contar sirven con cualquiera.
Faltantes	DUMMY_PLUG rellena con un valor de la misma clase que la columna. EQUIP no convierte a entero ni a booleano si quedan vacíos.
Archivos	ENTRY_PLUG falla si el archivo no existe. SEELE solo recibe tablas y solo escribe .csv.

6. Errores

Hay tres clases de error y todas dicen la línea y la columna, muestran la línea de código y ponen una flecha debajo del sitio exacto. Los mensajes de ANTLR vienen en inglés (*mismatched input*…), así que se traducen antes de mostrarlos. Cuando algo falla aparece el comandante Ikari:



```
ALERTA: sincronización fallida.
Nivel de contaminación mental alto, el EVA no arranca.

Error sintáctico en sin_01_falta_corchete.eva, línea 4, columna 1
  no se esperaba 'MONITOR'. Aquí debería ir ']' o ','.
      4 | MONITOR pilotos
        | ^

Gendo: "Corrige la línea 4, Shinji. O vete."
```

Un ejemplo de cada clase (sin repetir el dibujo):

Léxico

```
Error léxico en lex_01_caracter_invalido.eva, línea 3, columna 50
  el carácter '@' no pertenece al lenguaje.
      3 | altas = pruebas |> AT_FIELD donde sincronizacion @ 60
        |                                                  ^
```

Sintáctico

```
Error sintáctico en sin_02_etapa_inexistente.eva, línea 3, columna 20
  no se esperaba 'FILTRAR'. Aquí debería ir una etapa (TARGET, AT_FIELD, SYNC, SQUAD, MAGI...).
      3 | altas = pruebas |> FILTRAR donde sincronizacion > 60
        |                    ^
```

Semántico

```
Error semántico en sem_02_columna_inexistente.eva, línea 3, columna 35
  'sincronisacion' no es una columna de la tabla ni una variable. ¿Quisiste decir 'sincronizacion'?
      3 | altas = pruebas |> AT_FIELD donde sincronisacion > 60
        |                                   ^
```

Semántico

```
Error semántico en sem_03_tipos_incompatibles.eva, línea 3, columna 33
  tipos incompatibles: '*' necesita números y recibió una columna de texto.
      3 | raras = pruebas |> SYNC doble = piloto * 2
        |                                 ^
```

El error semántico solo aparece si el programa está bien escrito: primero se revisan todos los errores léxicos y sintácticos, y la ejecución se detiene en el primer error semántico.

7. Programas completos

En `ejemplos/` hay 4 programas que corren de principio a fin y 13 con errores a propósito. Estos dos muestran casi todo el lenguaje.

Resumen por piloto

```
# Limpia las pruebas y saca un resumen por piloto
NERV "Resumen por piloto"

pruebas = ENTRY_PLUG "datos/pruebas_sincronizacion.csv"

# Hay una prueba repetida y algunas sin dato
limpias = pruebas
    |> EJECT duplicados
    |> EJECT faltantes [sincronizacion]
    |> DUMMY_PLUG armonicos = 1.0
    |> SYNC indice = redondear(sincronizacion * armonicos, 1),
            apto = sincronizacion >= 40

por_piloto = limpias
    |> SQUAD por [piloto, unidad]
    |> MAGI pruebas = contar(),
            promedio = media(sincronizacion),
            mejor = maximo(sincronizacion),
            peor = minimo(sincronizacion),
            variacion = desviacion(sincronizacion)
    |> FORMATION por [promedio desc]

MONITOR por_piloto
SEELE por_piloto como "salidas/resumen_pilotos.csv"

# Estadisticas de cada columna
MONITOR limpias |> DIAGNOSTIC
```

```
MONITOR por_piloto (5 filas x 7 columnas)
piloto unidad  pruebas  promedio  mejor  peor  variacion
Kaworu EVA-02        1     98.00  98.00 98.00       nulo
Shinji EVA-01        9     67.18  93.50 55.20      12.39
 Asuka EVA-02        8     66.60  92.70 30.30      20.14
   Rei EVA-00       10     56.84  77.90 43.70       9.88
  Toji EVA-03        1     35.20  35.20 35.20       nulo

SEELE recibió el reporte salidas/resumen_pilotos.csv (5 filas)

MONITOR (9 filas x 9 columnas)
       columna    clase  validos  faltantes  distintos  media  desviacion  minimo  maximo
        prueba   numero       29          0         29  16.14        9.26    1.00   31.00
         fecha    texto       29          0         10   nulo        nulo    nulo    nulo
        piloto    texto       29          0          5   nulo        nulo    nulo    nulo
        unidad    texto       29          0          4   nulo        nulo    nulo    nulo
sincronizacion   numero       29          0         29  63.41       16.45   30.30   98.00
     armonicos   numero       29          0         24   1.00        0.22    0.62    1.38
   bateria_min   numero       29          0          3   4.28        0.88    3.00    5.00
        indice   numero       29          0         29  62.34       19.93   33.80  119.60
          apto booleano       29          0          2   nulo        nulo    nulo    nulo
```

El archivo `salidas/resumen_pilotos.csv` queda así:

```
piloto,unidad,pruebas,promedio,mejor,peor,variacion
Kaworu,EVA-02,1,98.0,98.0,98.0,
Shinji,EVA-01,9,67.1778,93.5,55.2,12.3947
Asuka,EVA-02,8,66.6,92.7,30.3,20.1384
Rei,EVA-00,10,56.84,77.9,43.7,9.8836
Toji,EVA-03,1,35.2,35.2,35.2,
```

Informe de combates

```
# El archivo de combates viene con ; y con nombres de columna incomodos
NERV "Informe de combates para el comandante"

combates = ENTRY_PLUG "datos/combates.csv" con {
    separador: ";"
    codificacion: "utf-8"
}
    |> CODENAME [`Angel` -> angel,
                 `Episodio` -> episodio,
                 `Unidad EVA` -> unidad,
                 `Piloto` -> piloto,
                 `Duracion (min)` -> minutos,
                 `Daño (%)` -> dano,
                 `Cable cortado` -> sin_cable]
    |> EQUIP sin_cable -> booleano

# Cuanto daño recibe cada unidad
por_unidad = combates
    |> SQUAD por [unidad]
    |> MAGI combates = contar(),
            dano_medio = media(dano),
            minutos_total = suma(minutos)
    |> FORMATION por [dano_medio desc]

MONITOR por_unidad
SEELE por_unidad como "salidas/dano_por_unidad.csv"

# La bateria interna dura 5 minutos: estos combates fueron los mas apretados
criticos = combates
    |> AT_FIELD donde sin_cable y minutos > 5
    |> SYNC riesgo = redondear(dano * minutos / 100, 2)
    |> TARGET [angel, unidad, piloto, minutos, riesgo]
    |> FORMATION por [riesgo desc]

MONITOR criticos

dano_general = media(combates.dano)
MONITOR dano_general

PATTERN_BLUE dano_general > 40 {
    SEELE criticos como "salidas/combates_criticos.csv"
} PATTERN_ORANGE {
    MONITOR "Sin novedad, las unidades aguantan"
}

# La grafica todavia no se dibuja, pero ya se revisa que este bien escrita
HOLOGRAM barras de por_unidad {
    eje_x: unidad
    eje_y: dano_medio
    titulo: "Daño promedio por unidad"
}
```

```
MONITOR por_unidad (3 filas x 4 columnas)
unidad  combates  dano_medio  minutos_total
EVA-01        10       52.00          74.60
EVA-00         7       47.43          55.80
EVA-02         8       43.62          69.10

SEELE recibió el reporte salidas/dano_por_unidad.csv (3 filas)

MONITOR criticos (6 filas x 5 columnas)
     angel unidad piloto  minutos  riesgo
   Israfel EVA-01 Shinji    14.00   10.22
   Bardiel EVA-02  Asuka    13.00    9.49
   Bardiel EVA-00    Rei    10.70    6.42
Sandalphon EVA-02  Asuka     5.40    4.10
  Armisael EVA-00    Rei     6.70    3.95
    Zeruel EVA-00    Rei    11.20    3.70

MONITOR dano_general: 48.04

SEELE recibió el reporte salidas/combates_criticos.csv (6 filas)

HOLOGRAM: gráfica de barras revisada, el dibujo queda para la fase 3
```

Los datos de `datos/` son de práctica: los nombres salen de la serie y las cifras son inventadas.

8. Cómo se usa

```
instalar.bat                                (crea el entorno, instala y corre las pruebas)
eva ejemplos\01_primera_prueba.eva          (ejecuta un programa)

python run.py programa.eva --tokens         (muestra los tokens)
python run.py programa.eva --arbol          (muestra el árbol en texto)
python run.py programa.eva --svg arbol.svg  (guarda el árbol como dibujo)
python run.py programa.eva --simbolos       (muestra la tabla de símbolos)
python run.py programa.eva --solo-analisis  (fase 1 sin ejecutar)

python -m pytest                            (corre las pruebas)
python regenerar.py                         (vuelve a generar lexer y parser)
```

Las rutas de los CSV se leen desde la carpeta donde se ejecuta el comando, por eso los ejemplos se corren parados en la raíz del proyecto.

Carpeta	Contenido
`grammar/`	Las dos gramáticas de ANTLR: lexer y parser.
`src/evangelion/`	El analizador (fase 1), el intérprete y la tabla de símbolos (fase 2), los errores, el árbol y los dibujos de la terminal.
`src/evangelion/generado/`	Lo que genera ANTLR. No se edita a mano.
`ejemplos/`	Programas válidos y, en `errores/`, programas que deben fallar.
`datos/`, `salidas/`	Los CSV de entrada y los reportes que escriben los programas.
`tests/`	Pruebas con pytest de las dos fases.

Lo que falta

Para la fase 3 queda dibujar las gráficas de `HOLOGRAM` con Matplotlib y montar el caso de estudio final. Tampoco hay funciones definidas por el usuario ni ciclos; por ahora no hicieron falta porque

las etapas ya trabajan sobre toda la tabla.